

NLP Program 3: Word Sense Disambiguation

Group 11: Alicia Maranville and Fred Solis

May 4, 2026

1 Overview

For the word sense disambiguation (WSD) task, we trained a classifier to predict which of two senses is being used. We adapted Pedersen’s Naive Bayes ensemble approach by training multiple Multinomial Naive Bayes classifiers over different context window sizes [1]. We trained a model for each word with varying hyperparameters and manually verified accuracy against the test sentences for *director*, *overtime*, and *rubbish*.

2 Method

2.1 Preprocessing

Since the corpus was simply a list of sentences, we conducted a small amount of preprocessing: converting text to lowercase, dropping punctuation, taking $\pm N$ tokens around the target and excluding the target from the context, and joining the context tokens with spaces to input into `CountVectorizer`. We removed stopwords for specific words as discussed in Section 2.3.

2.2 Algorithm

As previously stated, we followed the approach of Pederson to build an ensemble of classifiers for WSD [1]. The procedure is as follows:

1. For each training sentence, locate the target word and extract its surrounding context window of $\pm N$ tokens.
2. Represent the context as a bag of words using `CountVectorizer`.
3. Train a Multinomial Naive Bayes classifier with Laplace smoothing.
4. Repeat steps 1–3 for several different window sizes $N \in \{2, 5, 10, 25\}$ to produce four classifiers per word.
5. To test, average the predicted class probabilities across the four classifiers and pick the maximum as the predicted label.

Different senses may be disambiguated by features at various distances from the target word. The shorter windows account for immediate context (e.g., the phrase *film director* was used frequently for the second sense definition of *director*), while the wide windows account for context around the topics of the sentences (e.g., work or sports topics to distinguish between senses of *overtime*). Using multiple window sizes and combining classifiers allows for a more robust model.

Naive Bayes has been a strong baseline for WSD and outperforms other classifiers such as decision trees, perceptrons, and 3-nearest-neighbors on some tasks [2]. Pedersen’s ensemble approach performed statistically significantly better than the individual classifier on two words *line* and *interest*, which inspired us to try to replicate this methodology. The resulting models are extremely lightweight and load extremely quickly. Predictions also take fractions of a second.

2.3 Specific Differences Per Word

We tuned two parameters per word: Laplace smoothing and stopwords removal.

In Multinomial Naive Bayes, α controls how much probability is added to unseen words. The default value $\alpha = 1$ is meant to prevent zero probabilities, but it can reduce the impact of rare but useful contextual words, especially on our small dataset. So, we considered smaller smoothing values.

Additionally, we tested whether removing common English function words improves performance. We tuned this rather than always removing stopwords because we noticed in hurt performance for *rubbish* in particular: for example, *this is rubbish* often signals nonsense, and *the rubbish* often signals trash.

The chosen values are summarized in Table 1.

Table 1: Per word parameters.

Word	Window sizes N	Smoothing α	Stopwords removed
<i>director</i>	{2, 5, 10, 25}	0.1	no
<i>overtime</i>	{2, 5, 10, 25}	0.1	yes
<i>rubbish</i>	{2, 5, 10, 25}	0.5	no

2.4 Steps to Disambiguate a Sentence

To disambiguate a test sentence, the following steps are taken. We use *overtime* as an example with input "I worked overtime multiple times last week."

1. Load trained model: When `WSD_Test_overtime([sentence])` is called, the function unpickles the trained model in order to load a `NBEnsemble` object containing four pretrained classifiers (one per window size).
2. Tokenize sentence: The sentence is preprocessed and tokenized. The example tokenizes to:

`["i", "worked", "overtime", "multiple", "times", "last", "week"]`

3. Locate target word: Scan list for the first token that equals or starts with "overtime" and record its index, which is 2 in this example.
4. Retrieve context window for each ensemble part: For each window size $N \in \{2, 5, 10, 25\}$, take up to N tokens to the left of the target and up to N tokens to the right. With $N = 2$ on the example, the context becomes `i worked multiple times`. When the window is larger than the sentence, the context is the whole sentence without the target word.

5. Vectorize the context: Each context string is passed to the fitted `CountVectorizer`. Words never seen in training don't contribute anything. For *overtime*, we removed English stopwords, which are removed from the context before vectorizing.
6. Predict probabilities: For window size N the corresponding classifier computes, for each sense $s \in \{1, 2\}$

$$P(s \mid \text{context}) \propto P(s) \prod_{w \in V} P(w \mid s)^{c(w, \text{context})}, \quad (1)$$

where $c(w, \text{context})$ is the count of word w in the context and $P(w \mid s)$ is estimated from training counts with smoothing using the α for that word. The classifier's `predict_proba` method normalizes this and returns a two element probability vector.

7. Average probabilities across ensemble: The four probability vectors are averaged by element:

$$\bar{P} = \frac{1}{4}(P_{N=2} + P_{N=5} + P_{N=10} + P_{N=25}). \quad (2)$$

Unlike Pedersen's original ensemble that combined classifiers by majority vote, we used the average predicted probabilities to allow each window to contribute evidence in proportion to its confidence rather than forcing each to cast a binary vote.

8. Select the higher averaged probability: Take $\arg \max \bar{P}$ and return corresponding sense (1 or 2). For the example, the topic of *worked* should push each window's probabilities towards sense 1 (working extra hours).

References

- [1] Pedersen, T. (2000). *A Simple Approach to Building Ensembles of Naive Bayesian Classifiers for Word Sense Disambiguation*. Proceedings of the 1st Conference of the North American Chapter of the Association for Computational Linguistics (NAACL). arXiv:cs/0005006.
- [2] Mooney, R. J. (1996). *Comparative Experiments on Disambiguating Word Senses: An Illustration of the Role of Bias in Machine Learning*. Proceedings of the Conference on Empirical Methods in Natural Language Processing.